

Application: Bin Packing Problem (NP-Hard / Optimization Problem)

CO5 AT1

CSA0619 – DESIGN AND ANALYSIS OF ALGORITHMS

Submitted by

M Yasodha Krishna (192472100)

Under the Supervision of

Dr. F. Mary Harin Fernandez

ABSTRACT

The Bin Packing Problem is a well-known optimization problem in which items of different sizes must be packed into the minimum possible number of bins without exceeding the capacity of any bin. This problem occurs in many real-world applications such as warehouse storage, cloud computing, transportation, memory allocation, and resource management. Since finding the optimal solution is an NP-Hard problem, the computational complexity increases rapidly as the number of items grows. This report analyzes the constraints of bin capacity and item assignment, explains their influence on packing efficiency, develops a backtracking-based solution with pruning, and evaluates the complexity and performance of the approach. The pruning technique eliminates infeasible and unnecessary packing arrangements, improving the efficiency of the search for an optimal solution.

INTRODUCTION

Efficient resource utilization is an important problem in computer science and industry. In many situations, objects of different sizes must be placed into containers with limited capacity. The objective is to use the minimum number of containers while ensuring that no container exceeds its maximum capacity.

This problem is known as the Bin Packing Problem.

Given a collection of items and bins with a fixed capacity, each item must be assigned to exactly one bin. The total size of all items placed inside a bin must not exceed its capacity.

The Bin Packing Problem is classified as an NP-Hard optimization problem. As the number of items increases, the number of possible packing combinations grows rapidly. Therefore, exhaustive searching becomes inefficient for large inputs.

To solve the problem efficiently, techniques such as Backtracking, Branch and Bound, and heuristic algorithms such as First Fit Decreasing can be used. In this implementation, a backtracking approach with pruning is used to find an optimal packing.

PROBLEM STATEMENT

Given items of different sizes and bins with a fixed capacity, pack all items into the minimum possible number of bins without exceeding the capacity of any bin.

The solution must clearly satisfy the following constraints:

- Every item must be assigned to exactly one bin.
- The capacity of a bin must not be exceeded.
- All items must be packed.
- The total number of bins used must be minimized.

The objective is to develop a backtracking-based solution with pruning, analyze how

constraints affect packing efficiency, and discuss the NP-Hard complexity of the problem.

OBJECTIVES

- To understand the Bin Packing Problem.
- To define bin capacity and item assignment constraints.
- To minimize the total number of bins used.
- To implement a backtracking-based solution.
- To apply pruning to eliminate infeasible packings.
- To ensure that no bin exceeds its capacity.
- To analyze the effect of constraints on packing efficiency.
- To discuss the NP-Hard complexity of the optimization problem.

SYSTEM DESCRIPTION

The proposed system receives a set of items, where each item has a specific size. The system also defines a fixed capacity for every bin.

The algorithm attempts to place each item into an existing bin. Before placing an item, it checks whether the total size of the bin will remain within the allowed capacity.

If the item cannot be placed in any existing bin, a new bin is created.

The backtracking algorithm explores different possible arrangements of items. Pruning is used to stop exploring arrangements that:

- Exceed bin capacity.

- Already use more bins than the best solution found.
- Produce duplicate equivalent states.

The final output displays the minimum number of bins and the items assigned to each bin.

BIN PACKING CONSTRAINTS

1. Bin Capacity Constraint

The total size of items in a bin must not exceed its capacity.

$$\sum i \text{ item sizes} \leq \text{Bin Capacity}$$

For example, if the bin capacity is:

$$10$$

Then the total size of all items placed inside one bin must be less than or equal to 10.

2. Item Assignment Constraint

Every item must be assigned to exactly one bin.

An item cannot:

- Be assigned to multiple bins.
- Be left unpacked.

3. Optimization Constraint

The number of bins used should be minimized.

Minimize Number of Bins

BACKTRACKING APPROACH

Backtracking systematically explores possible solutions.

The process consists of the following steps:

Step 1:

Select an item.

Step 2:

Try placing the item into each existing bin.

Step 3:

Check whether the bin capacity constraint is satisfied.

Step 4:

If the item fits, place it in the bin and recursively process the next item.

Step 5:

If the placement does not lead to an optimal solution, undo the assignment.

Step 6:

Try another possible bin arrangement.

Step 7:

Store the best solution with the minimum number of bins.

PRUNING TECHNIQUES

Pruning improves the performance of backtracking by eliminating unnecessary searches.

The following pruning conditions are used:

Capacity Pruning

If:

$$\textit{Current Load} + \textit{Item Size} > \textit{Bin Capacity}$$

The item cannot be placed in that bin.

Therefore, that arrangement is rejected immediately.

Optimality Pruning

If the current number of bins is already greater than or equal to the best solution found, unnecessary exploration is avoided.

Symmetry Pruning

If two bins have the same current load, placing an item into either bin produces equivalent states.

Therefore, duplicate exploration can be avoided.

ALGORITHM

Step 1:

Start.

Step 2:

Read the item sizes.

Step 3:

Define the bin capacity.

Step 4:

Sort the items in decreasing order to improve packing efficiency.

Step 5:

Initialize the best solution.

Step 6:

Select the next item.

Step 7:

Try placing the item into each existing bin.

Step 8:

Check whether the bin capacity will be exceeded.

Step 9:

If valid, place the item and recursively process the next item.

Step 10:

Apply pruning when the current arrangement cannot improve the best solution.

Step 11:

If all items are packed, update the best solution if fewer bins are used.

Step 12:

Display the optimal bin allocation.

Step 13:

Stop.

IMPLEMENTATION

Q19 - Bin Packing Problem

Backtracking with Pruning

```
items = [7, 5, 6, 4, 2, 3]
```

```
bin_capacity = 10
```

```
# Sort items in decreasing order
```

```
items.sort(reverse=True)
```

```
best_solution = []
```

```
minimum_bins = len(items)
```

```
def bin_packing(index, bins):
```

```
    global best_solution
```

```
    global minimum_bins
```



```
# Pruning: current solution already uses
# more bins than the best solution
if len(bins) >= minimum_bins:
    return

# All items have been packed
if index == len(items):

    minimum_bins = len(bins)

    best_solution = [bin.copy() for bin in bins]

    return

item = items[index]

used_loads = set()

# Try placing the item in existing bins
for i in range(len(bins)):

    current_load = sum(bins[i])
```

```
# Symmetry pruning

if current_load in used_loads:

    continue

used_loads.add(current_load)


# Capacity constraint

if current_load + item <= bin_capacity:

    bins[i].append(item)

    bin_packing(index + 1, bins)


# Backtracking

bins[i].pop()


# Try creating a new bin

if len(bins) + 1 < minimum_bins:

    bins.append([item])
```

```
bin_packing(index + 1, bins)
```

```
bins.pop()
```

```
def main():
```

```
    global minimum_bins
```

```
    print("=" * 60)
```

```
    print("BIN PACKING PROBLEM")
```

```
    print("BACKTRACKING WITH PRUNING")
```

```
    print("=" * 60)
```

```
    print("\nItems:", items)
```

```
    print("Bin Capacity:", bin_capacity)
```

```
    # Initial upper bound:
```

```
    # each item can be placed in a separate bin
```

```
    minimum_bins = len(items) + 1
```

```
bin_packing(0, [])
```

```
print("\nOPTIMAL BIN ALLOCATION")
```

```
print("-" * 60)
```

```
for i, bin_items in enumerate(best_solution, start=1):
```

```
    print(
        "Bin", i,
        ":", bin_items,
        "| Total =", sum(bin_items)
    )
```

```
print("\nMinimum Number of Bins:", minimum_bins)
```

```
print("\nConstraints Satisfied:")
```

```
print("1. Every item is assigned exactly once.")
```

```
print("2. No bin exceeds capacity.")
```

```
print("3. Number of bins is minimized.")
```

```
print("=" * 60)
```

```
main()
```

SAMPLE INPUT

Items = [7, 5, 6, 4, 2, 3]

Bin Capacity = 10

SAMPLE OUTPUT

=====

=

BIN PACKING PROBLEM

BACKTRACKING WITH PRUNING

=====

=

Items: [7, 6, 5, 4, 3, 2]

Bin Capacity: 10

OPTIMAL BIN ALLOCATION

Bin 1 : [7, 3] | Total = 10

Bin 2 : [6, 4] | Total = 10

Bin 3 : [5, 2] | Total = 7

Minimum Number of Bins: 3

Constraints Satisfied:

1. Every item is assigned exactly once.
2. No bin exceeds capacity.
3. Number of bins is minimized.

=====

=

OUTPUT SCREENSHOT

```

>>> ===== RESTART: C:/NLP/CO5 AT1 DAA/source code.py =====
=====
BIN PACKING PROBLEM
BACKTRACKING WITH PRUNING
=====

Items: [7, 6, 5, 4, 3, 2]
Bin Capacity: 10

OPTIMAL BIN ALLOCATION
-----
Bin 1 : [7, 3] | Total = 10
Bin 2 : [6, 4] | Total = 10
Bin 3 : [5, 2] | Total = 7

Minimum Number of Bins: 3

Constraints Satisfied:
1. Every item is assigned exactly once.
2. No bin exceeds capacity.
3. Number of bins is minimized.
=====
>>> |

```

CONSTRAINT ANALYSIS

The constraints directly influence packing efficiency.

Bin Capacity

A smaller bin capacity may require more bins.

For example:

Items: [7, 6, 5, 4]

With capacity 10, some items can be combined.

With capacity 7, fewer combinations are possible, resulting in more bins.

Item Sizes

Large items reduce the available space in bins and make efficient packing more difficult.

Small items can often fill unused spaces and improve utilization.

Item Assignment

Every item must be assigned exactly once. This constraint prevents the algorithm from ignoring difficult-to-pack items.

COMPLEXITY ANALYSIS

The Bin Packing Problem is an NP-Hard optimization problem.

The backtracking algorithm may need to explore many possible combinations of assigning items to bins.

Worst-Case Time Complexity

The number of possible arrangements grows exponentially.

Therefore, the worst-case complexity can be expressed as:

$$O(2^n)$$

or exponential in nature, depending on the branching possibilities.

Space Complexity

The recursion and bin storage require approximately:

$$O(n)$$

excluding the storage required for the solution itself.

WHY IS BIN PACKING NP-HARD?

The Bin Packing Problem requires finding the optimal arrangement of items.

As the number of items increases:

- The number of possible combinations increases rapidly.
- Checking every possible arrangement becomes expensive.
- Finding the minimum number of bins cannot generally be done efficiently using a known polynomial-time algorithm.

Therefore, the optimization version of the Bin Packing Problem is classified as NP-Hard.

This is why practical systems often use:

- Backtracking
- Branch and Bound
- First Fit
- Best Fit
- First Fit Decreasing
- Approximation algorithms

for large-scale problems.

PERFORMANCE ANALYSIS

The backtracking solution provides an exact solution for small and medium-sized inputs.

Advantages of Backtracking

- Finds an optimal solution.
- Explores different packing combinations.

- Ensures all constraints are satisfied.

Effect of Pruning

Pruning improves performance by avoiding:

- Bins that exceed capacity.
- Solutions worse than the current best solution.
- Duplicate arrangements with identical bin states.

This reduces the number of unnecessary recursive calls.

SCALABILITY ANALYSIS

For a small number of items, backtracking can efficiently find the optimal solution.

However, as the number of items increases:

- The search space grows exponentially.
- Execution time increases significantly.
- Finding the exact optimal solution becomes difficult.

For very large datasets, heuristic algorithms are generally preferred because they provide fast solutions, although they may not always produce the absolute minimum number of bins.

APPLICATIONS

The Bin Packing Problem is used in:

- Warehouse storage.
- Cloud resource allocation.
- Memory allocation.

- Transportation and logistics.
- Container loading.
- Server resource management.
- Task scheduling.
- Data center optimization.
- Manufacturing systems.
- File storage systems.

ADVANTAGES

- Finds an efficient packing arrangement.
- Minimizes the number of bins.
- Ensures capacity constraints are satisfied.
- Backtracking can find an optimal solution.
- Pruning reduces unnecessary computation.
- Applicable to many real-world optimization problems.

LIMITATIONS

- The problem is NP-Hard.
- Exact solutions become slow for large numbers of items.
- Backtracking may explore many combinations.
- Memory and execution time can increase significantly.
- Heuristic solutions may not always produce the optimal result.

RESULT ANALYSIS

The Bin Packing Problem was successfully solved using a Backtracking approach with pruning.

The algorithm ensured that:

- Every item was assigned exactly once.
- No bin exceeded the capacity of 10.
- The number of bins was minimized.

For the given input, the algorithm found an optimal solution using 3 bins.

The pruning techniques improved efficiency by eliminating infeasible and duplicate packing arrangements. This demonstrates how constraint checking and optimization techniques can significantly reduce the search space in NP-Hard problems.

CONCLUSION

The Bin Packing Problem was successfully analyzed and implemented as an NP-Hard optimization problem using Backtracking with Pruning. The solution assigns items of different sizes into bins while ensuring that no bin exceeds its maximum capacity.

The algorithm searches for an arrangement that uses the minimum number of bins. Pruning techniques eliminate infeasible packings, duplicate states, and solutions that cannot improve the current best result.

Although the backtracking approach can produce an optimal solution, its computational complexity grows rapidly as the number of items increases. Therefore, exact methods are suitable for smaller datasets, while heuristic and approximation algorithms are more practical for large-scale applications.

The implementation demonstrates the importance of constraints, optimization, backtracking, and pruning in solving NP-Hard problems.

REFERENCES

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, Introduction to Algorithms, MIT Press.
2. Ellis Horowitz and Sartaj Sahni, Fundamentals of Computer Algorithms.
3. Michael R. Garey and David S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness.
4. Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman, The Design and Analysis of Computer Algorithms.
5. Course Notes on Design and Analysis of Algorithms